

Lab2 coroutinelab

姓名：张芝源 学号：2024010701 班级：计42

首先，在vscode里安装remote-ssh插件，点击左下角><，选择Connect to Host..., Add New SSH Host..., 输入ssh -p 31122 2024010701@202.205.2.250，就可以连接上了，然后把本地的文件夹拖拽过来，就可以在此窗口里享受远程linux服务器了。在本地用typora编辑md，最后拖拽大法拖上去就ok

Task1

所谓协程切换，就是有两个进程时，从进程A yield()(切出)到进程B时，发生了存档进程A，读档进程B的过程

如果存档位置是A和B的话，那就是先把现在的CPU寄存器保存到A，再把B的数据读进CPU寄存器，此时，程序就切到了B

各个文件的作用：

lib/context.S:写汇编指令，把寄存器搬进内存（保存），再从内存搬回寄存器（恢复）。

inc/context.h (数据结构):这里定义了存档长什么样。

inc/common.h (操作入口): 这里是 yield 函数。任务：当你想暂停时，调用上面的汇编函数，把自己存起来，切换回调度器。

inc/coroutine_pool.h (调度器): 这里是管理员。它决定下一个读谁的档

在context.S中：

A. 保存返回地址到(RIP)

根据函数定义 `void coroutine_switch(uint64_t *save, uint64_t *restore)`

%rdi是第一个参数（保存位置，也就是存档A的位置）

%rsi是第二个参数（恢复位置，也就是存档B的位置）

rip指向CPU下一条要执行的指令在内存中的地址

rsp是栈指针（指向栈中最顶层元素）

在64位系统中，指针是8位的，

```
enum class Registers : int {
    RAX = 0,
    RDI,
    // ... 中间省略 ...
    R15, // 第 14 号
    RIP, // 第 15 号 (这就是我们要存的位置)
    RegisterCount
};
```

所以是从数组开头（%rdi）往后数120个字节，把CPU要执行的下一个指令存到rip

```
movq (%rsp), %rax
movq %rax, 120(%rdi)
```

完整逻辑：因为函数调用（call），所以CPU把下一条指令地址存在了栈顶（%rsp），把它读出来放到%rax，再放到%rip，让%rip等于CPU下一条指令的地址（也就是函数的返回地址）

但是不可以直接movq (%rsp), 120(%rdi)，不可以从内存到内存

120(%rdi)是RDI指向的数组的第120个位置，因为RDI是第一个参数，也就是保存的起始位置！

B. 保存栈指针

当你在C++里调用coroutine_switch时，或者在汇编里执行call指令时，CPU先把下一条指令的地址压入栈顶，导致%rsp减少了8字节（因为栈向下增长）

所以先用leaq 8(%rsp), %rax，将当前%rsp的值+8，算出来结果放到%rax寄存器中
然后用movq %rax, 64(%rdi)，也就是把栈顶地址放到RSP寄存器中

C. 保存 Callee-Saved 寄存器 (RBX, RBP, R12-R15)

```
movq %rbx, 72(%rdi)
movq %rbp, 80(%rdi)
movq %r12, 88(%rdi)
movq %r13, 96(%rdi)
movq %r14, 104(%rdi)
movq %r15, 112(%rdi)
```

D. 恢复called-saved寄存器

```
movq 72(%rsi), %rbx
movq 80(%rsi), %rbp
movq 88(%rsi), %r12
movq 96(%rsi), %r13
movq 104(%rsi), %r14
movq 112(%rsi), %r15
```

E. 恢复栈指针 (RSP)

```
movq 64(%rsi), %rsp
```

F. 恢复RIP

将目标 RIP 压入栈顶，这样执行 ret 指令时就会跳转到那里

```
pushq 120(%rsi)
```

全部代码：

```
.global coroutine_entry
coroutine_entry:
    movq %r13, %rdi
    callq *%r12

.global coroutine_switch
coroutine_switch:
```

```
# -----
```

```

# Task 1: 上下文切换

# 函数原型: void coroutine_switch(uint64_t *save, uint64_t *restore)

# 参数: %rdi = save 数组指针, %rsi = restore 数组指针

# -----

# 1. 保存当前执行流的上下文到 save (%rdi)

# A. 保存返回地址到(RIP)
# 当调用此函数时, 返回地址已经被压入栈顶 (%rsp)。
# 我们将其读出并保存到 save 数组的 RIP 位置 (索引 15, 偏移 120)
movq (%rsp), %rax
movq %rax, 120(%rdi)

# B. 保存栈指针 (RSP)
# 我们需要保存的是"调用者看起来的栈顶"。
# 由于 ret 指令会弹出返回地址, 所以逻辑上的 RSP 应该是当前 %rsp + 8
leaq 8(%rsp), %rax
movq %rax, 64(%rdi)

# C. 保存 Callee-Saved 寄存器 (RBX, RBP, R12-R15)
# 根据 Registers 枚举顺序
movq %rbx, 72(%rdi)
movq %rbp, 80(%rdi)
movq %r12, 88(%rdi)
movq %r13, 96(%rdi)
movq %r14, 104(%rdi)
movq %r15, 112(%rdi)

# -----

# 2. 从 restore (%rsi) 恢复上下文

# D. 恢复 Callee-Saved 寄存器
movq 72(%rsi), %rbx
movq 80(%rsi), %rbp
movq 88(%rsi), %r12
movq 96(%rsi), %r13
movq 104(%rsi), %r14
movq 112(%rsi), %r15

# E. 恢复栈指针 (RSP)
movq 64(%rsi), %rsp

# F. 恢复执行流 (RIP)
# 将目标 RIP 压入栈顶, 这样执行 ret 指令时就会跳转到那里
pushq 120(%rsi)

# 3. 跳转
ret

```

在common.h和context.h中:

yield()函数中, 此刻跑的是协程, 协程要下线了, 存档: 把当前 CPU (协程的状态) 保存进 `callee_registers`; 读档: 从 `caller_registers` 读取调度器的状态, 恢复到 CPU

resume()函数中: 此刻跑的是调度器, 调度器要下线了, 存档: 把当前CPU (调度器的状态) 保存进 `caller_registers`;读档: 从 `callee_registers` 读取协程刚才暂停时的状态, 恢复到CPU

`callee_registers`: 这是**协程 (Coroutine)** 专用的内存。不管谁在跑, 只要涉及协程的数据, 都往这里存取。

`caller_registers`: 这是**调度器 (Scheduler)** 专用的内存。

也就是说从协程A转到协程B时, 要先进过一次调度器

explanation from gemini:

1. 协程 A 的视角: 存档 (Yield)

当协程 A 运行到一半调用 `yield()` 时:

- 动作: `coroutine_switch(A->callee, A->caller)`
- 结果: 协程 A 的当前进度 (PC 指针、栈指针、寄存器) 被“冻结”并塞进了它自己的 `callee_registers` 里。
- 然后: CPU 离开了 A, 回到了调度器手中。

(此时, A 就像一个被按了暂停键的游戏, 存档文件就放在 `A->callee_registers` 里, 静静地等着。)

2. 调度器的视角: 切歌 (Schedule)

调度器拿回控制权后, 它可能会先去跑协程 B、协程 C..... 在这个过程中, A 的存档一直安全地躺在 A 的柜子里, 没人动它。

3. 未来的某一刻: 读档 (Resume A)

等调度器 (可能跑了一圈后) 再次决定: “轮到协程 A 了!” 它会调用 `A->resume()`:

- 动作: `coroutine_switch(A->caller, A->callee)`
- 结果: CPU 打开 `A->callee_registers`, 读出当初 A 存进去的数据。
- 奇迹发生: A 醒来了! 它发现自己还在当初调用 `yield()` 的那一行代码之后, 好像刚才的一切都没发生过一样。

实现调度器逻辑 (coroutine_pool.h) :

```
// [inc/coroutine_pool.h]

/**
 * @brief 以协程执行的方式串行并同时执行所有协程函数
 */
void serial_execute_all() {
    is_parallel = false;
    g_pool = this;

    // 循环, 直到所有协程都执行完毕
```

```

while (true) {
    bool all_finished = true; // 假设大家都跑完了

    // 遍历每一个协程
    for (int i = 0; i < coroutines.size(); i++) {
        auto context = coroutines[i];

        // 如果发现有人还没跑完
        if (!context->finished) {
            all_finished = false; // 那就说明还得继续循环

            // Task 1 逻辑: 只要没结束, 就认为是 Ready 的, 直接调度
            // (Task 2 我们会在这里加 sleep 的判断, 现在先不管)
            if (context->ready) {
                context_id = i; // 记录当前是谁在跑 (给 yield 用)
                context->resume(); // 切进去!
            }
        }
    }

    // 如果遍历一圈发现所有人都 finished 了, 那就收工
    if (all_finished) break;
}

// 清理内存
for (auto context : coroutines) {
    delete context;
}
coroutines.clear();
}

```

运行程序, 输出:

```

2024010701@ics24:~/coroutinelab$ ./bin/sample
in show(): 0
in show(): 0
in show(): 1
in show(): 1
in show(): 2
in show(): 2
in show(): 3
in show(): 3
in show(): 4
in show(): 4
in main(): 0
in main(): 0
in main(): 1
in main(): 1
in main(): 2
in main(): 2
in main(): 3
in main(): 3
in main(): 4
in main(): 4

```

task1的额外要求:

1)绘制出在协程切换时, 栈的变化过程

协程切换时, 进行 `coroutine_switch` 函数

(1) CPU 进行协程 A, 刚刚调用了 `call coroutine_switch`:

[协程 A 的栈]

(A 的局部变量)

Return Address (RIP-A) ← A 的逻辑栈顶 (old RSP)
(压入 `call` 指令)

← 当前 `%rsp` 指向

动作: 读取 (`%rsp`), 得到 RIP-A, 存入 `A → context rip`

计算 `%rsp + 8`, 得到 old RSP, 存入 `A → context rsp`

保存其他 `callee-saved` 寄存器, 得到 `A → context`

(2) 执行 `movq 04(%rsi), %rsp`. 此时的 RSP 从 A 的栈瞬间跳到 B 的栈

[协程 A 的栈]

RIP-A 残留数据

[协程 B 的栈]

← 切换后的 `%rsp` 指向这里.
也是 B 上次 `call` 时保存的 RSP

[`%rsp`] —————→

(3) 执行 `pushq 120(%rsi)` 将目标 RIP 压栈

此时的 `RSP`: 在 B 的栈上向下移动, 为 `ret` 指令做准备

[协程 B 的栈]

! B 的局部变量 !

Return Address(RIP-B) $\leftarrow$ `pushq` 指令压入
 $\leftarrow$ 当前 `%rsp` 指向这里

动作: 恢复其他寄存器从 B $\rightarrow$ context

`pushq RIP-B`: 将 B 上次暂停的代码地址压入栈顶

`ret` 指令: CPU 弹出栈顶的 `RIP-B` 赋值给 `%rip`

结果: CPU 跳转到 `RIP-B` 继续执行, 且此时 `%rsp` 指向 B 的栈帧

2) 结合源代码, 解释协程是如何开始执行的, 包括 `coroutine_entry` 和 `coroutine_main` 函数以及初始的协程状态:

协程切换的核心在于栈指针寄存器 (`%rsp`) 的替换。在 `coroutine_switch` 汇编函数执行期间, CPU 的视角从“前一个协程的栈”瞬间转移到了“后一个协程的栈”。

这个过程可以清晰地分为三个步骤, 如下图所示:



协程的第一次运行 (冷启动) 是一个特殊情况, 因为它的栈里没有“上次保存的记录”。我们需要在初始化时人工构造一个初始状态

首先是构造初始状态 (Basic Context)

在 `basic_context` 的构造函数中, 我们手动填充了 `callee_registers` 数组, 将 `RIP` 设置为汇编函数 `coroutine_entry` 的地址。这样, 当调度器第一次 `resume` 这个协程时, `ret` 指令会直接跳转到这里。由于标准的参数传递寄存器 (`RDI`, `RSI`) 在切换过程中会被覆盖, 我们利用了 `R12` 和 `R13` 这两个“被调用者保存寄存器”来携带启动所需的参数: `R12` 保存了 C++ 入口函数 `coroutine_main` 的地址。 `R13` 保存了 `this` 指针 (即协程对象本身)

当 CPU 跳转到 `coroutine_entry` 后, 它承担了“适配器”的角色。

因为 C++ 函数调用约定要求第一个参数必须放在 `%rdi` 寄存器中，而我们的 `this` 指针此刻还藏在 `%r13` 里。所以 `coroutine_entry` 执行了以下操作：

1. `movq %r13, %rdi`：把 `this` 指针移动到第一个参数的位置。
2. `callq *%r12`：调用 `coroutine_main` 函数。

`coroutine_main`：

最终进入了 `coroutine_main` 函数。它的逻辑非常清晰：

1. **执行用户任务**：调用 `context->run()`，这才是真正执行用户提供的函数（例如本实验中的 `show`）。
2. **处理结束**：当 `run()` 返回时，说明任务完成了。此时将 `finished` 设为 `true`。
3. **交还控制权**：最后，必须显式调用 `coroutine_switch` 切回调度器。这是因为协程栈是独立的，不能直接 `return` 回到主线程，必须通过上下文切换手动跳回去。

Task2

首先修改 `inc/common.h` 的 `sleep` 函数

逻辑：

记录当前时间（开始 `sleep` 的时间）。

定义一个 Lambda 函数 `ready_func`，用来计算“现在是不是该醒了”。

把协程标记为 `ready = false`（没空）。

调用 `yield()` 主动切出。

```
void sleep(uint64_t ms) {
    if (g_pool->is_parallel) {
        auto cur = get_time();
        while (
            std::chrono::duration_cast<std::chrono::milliseconds>(get_time() - cur)
                .count() < ms)
            ;
    } else {
        // --- Task 2 ---

        // 1. 获取当前协程
        auto context = g_pool->coroutines[g_pool->context_id];

        // 2. 记录入睡时间
        auto start_time = get_time();

        // 3. 唤醒条件 (Lambda表达式)
        // 调度器会不断调用这个函数，看是不是时间到了
        context->ready_func = [start_time, ms]() {
            auto now = get_time();
            auto elapsed = std::chrono::duration_cast<std::chrono::milliseconds>(now -
start_time).count();
            return elapsed >= ms;
        };
    }
}
```



```

};

// 4. 标记为"正在睡觉" (Not Ready)
context->ready = false;

// 5. 切出, 把 CPU 让给别的进程
yield();
}
}

```

然后修改inc/coroutine_pool.h

```

void serial_execute_all() {
    is_parallel = false;
    g_pool = this;

    while (true) {
        bool all_finished = true;

        for (int i = 0; i < coroutines.size(); i++) {
            auto context = coroutines[i];

            if (!context->finished) {
                all_finished = false; // 只要还有一个没完, 就不能退出大循环

                // Task 2逻辑: 检查睡眠是否结束
                // 如果协程是不可用的(!ready), 我们看看它是不是该醒了
                if (!context->ready) {
                    // 调用刚才在 sleep 里写的 lambda 函数
                    if (context->ready_func && context->ready_func()) {
                        context->ready = true; // 醒了! 标记为 Ready
                    }
                }
                // 只有 Ready 的协程才能被 resume
                if (context->ready) {
                    context_id = i;
                    context->resume();
                }
            }
        }

        if (all_finished) break;
    }

    for (auto context : coroutines) {
        delete context;
    }
    coroutines.clear();
}

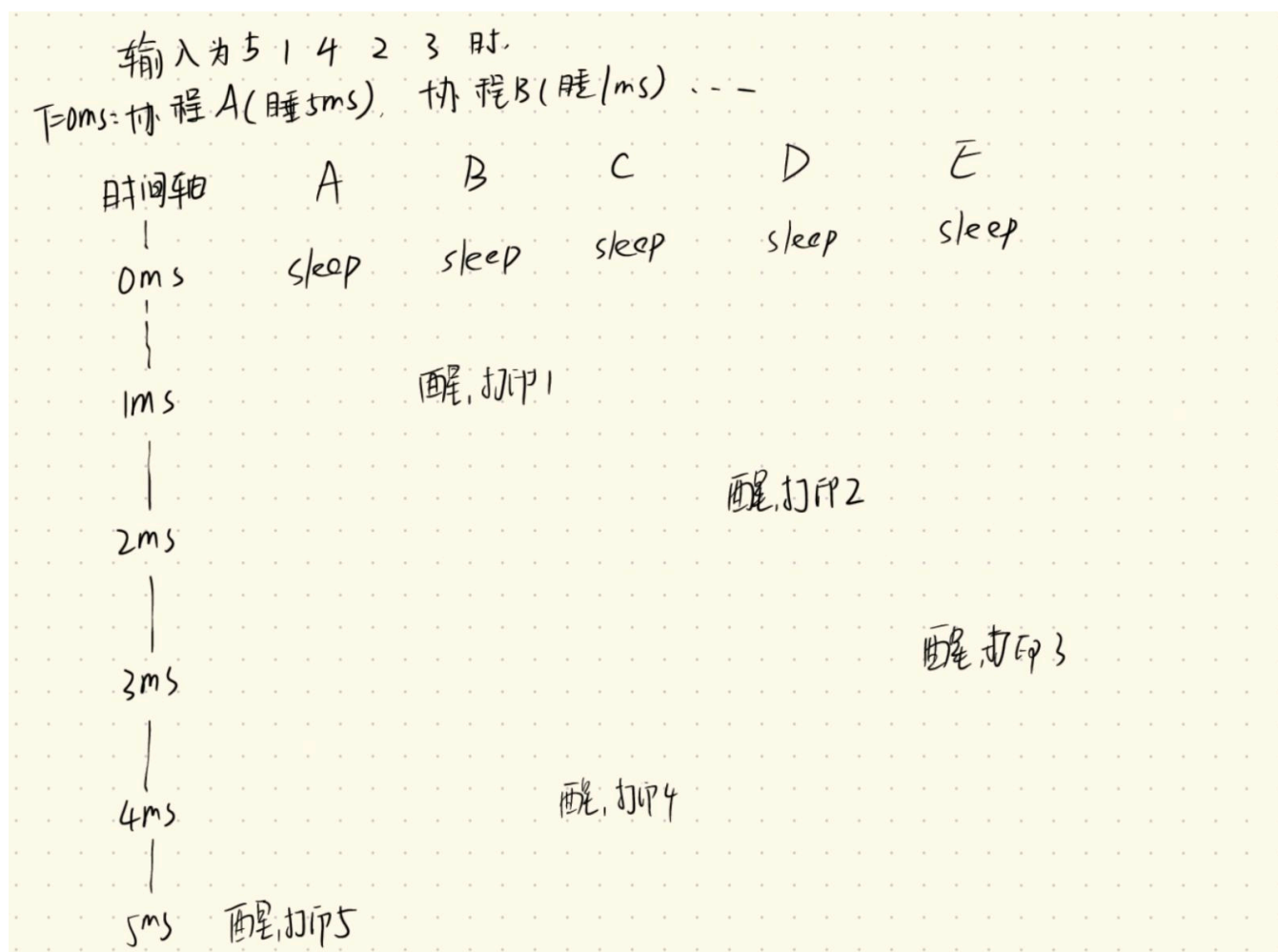
```

实验结果

```
2024010701@ics24:~/coroutinelab$ ./bin/sleep_sort
5
5 1 4 2 3
1
2
3
4
5
```

额外要求:

1) 绘制时间线



2) 现在的做法其实效率不高。因为调度器是轮询 $O(N)$ ，不管协程要睡多久，调度器每次循环都要把所有的人问一遍“你醒了吗”。

可以用一个优先队列（最小堆）来管理睡觉的协程。

把所有睡觉的协程放进堆里，按照唤醒时间排序。最早醒的放在堆顶。

调度器每次不需要遍历所有人，只需要看堆顶的那一个。

如果堆顶的时间都没到，那后面的人肯定也没到，就不用检查了。

如果堆顶时间到了，就把它取出来放到就绪队列里，然后再看下一个堆顶。

Task3

在大数组里做二分查找，内存访问是非常随机的，极易容易发生 Cache Miss（缓存未命中）。CPU 从内存读数据需要很久。可以采用协程做法，算坐标 -> 发出预取指令 -> 切到下一个协程（让 CPU 去算别人的坐标） -> ... -> 切回来时数据已经在缓存里了 -> 读内存 -> 比较。

所以修改lookup_coroutine函数

```
void lookup_coroutine(const uint32_t *table, size_t size, uint32_t value,
                     uint32_t *result) {
    size_t low = 0;
    while ((size / 2) > 0) {
        size_t half = size / 2;
        size_t probe = low + half;

        // TODO: Task 3
        // 1. 告诉 CPU: 我马上要用 table[probe] 这个地址的数据了，帮我把它从内存拉到缓存里来
        // 参数说明: addr, rw(0=读), locality(0=无时间局部性，因为二分查找很难再次访问同一个位置)
        __builtin_prefetch(&table[probe], 0, 0);

        // 2. 切出：在数据从内存加载到缓存的这段漫长时间里，把 CPU 让给别的协程去干活
        yield();

        uint32_t v = table[probe]; // 这时候数据大概率已经在 L1/L2 缓存了，读取速度飞快
        if (v <= value) {
            low = probe;
        }
        size -= half;
    }
    *result = low;
}
```

输出：

```
● 2024010701@ics24:~/coroutinelab$ ./bin/binary_search
Size: 4294967296
Loops: 1000000
Batch size: 16
Initialization done
naive: 1474.68 ns per search, 46.08 ns per access
coroutine batched: 1294.04 ns per search, 40.44 ns per access
```

额外要求：汇报性能的提升效果：

测试环境与参数：

测试数据规模：4GB (Size: 4294967296 bytes)

循环次数 (Loops): 1,000,000 次

Batch Size: 16

测试场景：在大内存数组中进行随机二分查找。

性能数据对比：

测试项	平均每次查找耗时	平均每次内存访问耗时
Naive (基准)	1474.68	46.08
Coroutine (优化)	1294.04	40.44
提升幅度	+12.25%	+12.24%

Batch=16 (1294 ns): 能有效掩盖内存延迟。

Batch=64 (约 2362 ns): 性能倒退。 因为过多的协程导致 L1 指令缓存污染（Cache Pollution）以及频繁切换带来的开销超过了预取带来的收益。

感想

完成本实验的过程中，查阅了gemini帮助我理清思路（实在是太难一下子理解了T_T）

总结和感想：还是有空多看看CSAPP吧.....